

Bash Tutorial Exercises

Katherine Wolf

2026-08-31

1. Make a variable called `mypython` that contains the path to Python on your machine. You shouldn't need to manually type the path.

```
mypython=$(which python) # creates variable with the path
echo $mypython            # prints the path
```

`/home/kathe/opt/miniforge3/bin/python`

2. Construct a variable that has the value `<username>@<machinename>` using existing environment variables and the `hostname` utility.

I wasn't sure if the question intended the value in the variable to include the less-than and greater-than symbols, so I solved the problem both ways. (My laptop's name is "tris" after the character Tris in Divergent. Don't judge me, LOL.)

Excluding the less-than and greater-than symbols:

```
ghost_in_the_machine="$(whoami)@$(hostname)" # create variable
echo $ghost_in_the_machine                  # print value of variable
```

`kathe@tris`

Including the less-than and greater-than symbols:

```
ghost_in_the_machine="<$(whoami)>@<$(hostname)>" # create variable
echo $ghost_in_the_machine                      # print value of variable
```

`<kathe>@<tris>`

3. Figure out how to use the `mkdir` command to create the following directory structure in one short command:

```
temp
  proj1
    code
    data
  proj2
    code
    data
  proj3
    code
    data
```

```
mkdir -p temp/proj{1..3}/{code,data}
```

4. How would you count the number of lines in an input file, say, a data file?

This code presumes a file exists titled `datafile.txt` in the current working directory, structured as follows:

```
1 kitten 62234 26342
2 puppy 62314 26212
3 piglet 23314 92312
4 lamb 23344 84312
5 foal 82344 42312
6 calf 42325 23482
```

Code to count the lines:

```
wc -l datafile.txt    # -l modifier tells the wc function to count the number of new lines
```

```
6 datafile.txt
```

5. Print the first three lines of a file to the screen. Now print just the third line to the screen.

Using the same file, the first three lines:

```
head -n 3 datafile.txt
```

```
1 kitten 62234 26342
2 puppy 62314 26212
3 piglet 23314 92312
```

Third line only:

```
sed -n '3p' datafile.txt
```

```
3 piglet 23314 92312
```

6. Put the third line of a file in a new file.

```
sed -n '3p' datafile.txt > newfile.txt    # put the third line into a new file
cat newfile.txt                          # print new file to prove that it worked
```

```
3 piglet 23314 92312
```

7. Now add the fifth line of the file to that same file from the previous problem.

```
sed -n '5p' datafile.txt >> newfile.txt    # add the fifth line to the prior file
cat newfile.txt                          # print new file to prove that it worked
```

```
3 piglet 23314 92312
5 foal 82344 42312
```

8. Extract the Australia data from the cpds.csv dataset and put it in a file called cpds_australia.csv. It's OK if you do this in a straightforward way and it might fail if 'Australia' is present in an unexpected column.

```
grep Australia cpds.csv > cpds_australia.csv    # extract the Australia data to another file
cat cpds_australia.csv                          # proof that Australia is all that remains
```

1960,"Australia",95.5,NA,NA,1.42
1961,"Australia",95.3,NA,-0.07,2.79
1962,"Australia",95.3,23.17,5.71,2.63
1963,"Australia",95.7,23.01,6.1,2.12
1964,"Australia",95.7,22.88,6.28,1.15
1965,"Australia",95.7,24.9,4.97,1.15
1966,"Australia",95.1,26.06,2.9,1.6
1967,"Australia",95.1,26.35,7.25,1.88
1968,"Australia",95.1,26.52,5.3,1.82
1969,"Australia",95,25.05,6.29,1.82
1970,"Australia",95,25.23,6.96,1.65
1971,"Australia",95,25.55,3.85,1.91
1972,"Australia",95.4,27.34,2.32,2.61
1973,"Australia",95.4,26.03,4.39,2.31
1974,"Australia",95.4,28.99,1.58,2.67
1975,"Australia",95.4,31.64,2.35,4.9
1976,"Australia",95.4,33.81,3.92,4.77
1977,"Australia",95,34.86,1.45,5.63
1978,"Australia",95,34.59,2.03,6.43
1979,"Australia",95,32.97,4.65,6.26
1980,"Australia",94.4,32.94,2.84,6.11
1981,"Australia",94.4,33.11,4.07,5.78
1982,"Australia",94.4,35.79,-0.39,7.16
1983,"Australia",94.6,37.31,-0.17,9.96
1984,"Australia",94.2,38.81,6.55,8.99
1985,"Australia",94.2,39.72,5.22,8.26
1986,"Australia",94.2,39.43,2.24,8.08
1987,"Australia",93.8,37.22,5.17,8.11
1988,"Australia",93.8,34.07,4.42,7.23
1989,"Australia",93.8,32.56,4.26,6.17
1990,"Australia",95.3,33.38,1.52,6.93
1991,"Australia",95.3,35.89,-1.18,9.59
1992,"Australia",95.3,37.01,2.96,10.8
1993,"Australia",95.8,36.3,4.08,10.9
1994,"Australia",95.8,36.15,4.76,9.75
1995,"Australia",95.8,35.83,3.58,8.49
1996,"Australia",95.8,35.17,3.9,8.55
1997,"Australia",95.8,34.37,3.9,8.45
1998,"Australia",95,33.73,5.19,7.72
1999,"Australia",95,33.53,4.07,6.93
2000,"Australia",95,33.94,3.14,6.28
2001,"Australia",94.9,34.47,2.76,6.75
2002,"Australia",94.9,33.83,3.78,6.36

```
2003,"Australia",94.9,33.31,3.35,5.94
2004,"Australia",94.3,33.77,3.76,5.4
2005,"Australia",94.3,33.29,3.28,5.04
2006,"Australia",94.3,32.98,2.6,4.79
2007,"Australia",94.8,32.96,4.89,4.38
2008,"Australia",94.8,33.9,2.28,4.25
2009,"Australia",94.8,36.58,1.45,5.59
2010,"Australia",93.2,36.26,2.43,5.23
```

9. Find all the lines in a file that do not contain a comma. (You might use this to look for anomalies in a CSV file.)

```
type python
```

```
python is /home/kathe/opt/miniforge3/bin/python
```

10. Write shell code that creates files file1.txt, file2.txt, file3.txt, etc., with the word 'blah' as the only line in each file.

```
type python
```

```
python is /home/kathe/opt/miniforge3/bin/python
```